

MODULE 12 · DOMAIN 2 - DESIGN FOR NEW SOLUTIONS

Infrastructure as Code

Domain 2 begins. CloudFormation appears in 8% of the question bank.

DOMAIN 2 of 4 EXAM WEIGHT 29% MODULE 12 of 44

The template is not the valuable part. The **stack** is, because it remembers what it created.

The deployment safety chain

- ◆ A **change set** shows what an update will do before it does it, including which resources will be replaced.
- ◆ **Drift detection** finds resources changed outside CloudFormation, which the next deployment would overwrite.
- ◆ **Stack policies** deny updates to named resources, protecting a database from a careless template change.
- ◆ Failed updates roll back automatically, and **rollback triggers** can watch CloudWatch alarms during deployment.

ON THE EXAM

Replacement is the word to hunt for in a change set. It means a new resource and a deleted one, which for a database means data loss.

Controlling what happens on delete

- ◆ **DeletionPolicy: Retain** keeps a resource when the stack is deleted. Snapshot takes a final snapshot first.
- ◆ **UpdateReplacePolicy** does the same when an **update** would replace the resource. They are separate settings.
- ◆ Setting only DeletionPolicy leaves you exposed on updates, which is where most accidental data loss happens.
- ◆ **Termination protection** on the stack itself stops the whole stack being deleted.

ON THE EXAM

Deletion and replacement are different events with different settings. Databases need both policies, plus termination protection.

Nested stacks or cross-stack references?

NESTED STACKS

One lifecycle, many templates

- Child stacks owned by a parent
- Deployed and deleted together
- Good for reusable components
- Update the parent to change a child

CROSS-STACK REFERENCES

Separate lifecycles

- Export an Output, import elsewhere
- Stacks deploy independently
- Good for shared network or IAM
- An export cannot change while imported

THE TIEBREAKER

Ask whether the pieces live and die together. Shared infrastructure with its own lifecycle needs exports, not nesting.

StackSets across the organisation

- ◆ **One operation deploys to many accounts and Regions.** Target whole OUs, not lists of accounts.
- ◆ **Service-managed permissions** use Organizations and need no roles per account. Self-managed means you create them.
- ◆ **Automatic deployment** means an account joining a target OU receives the stack without anyone acting.
- ◆ Delegate administration to a member account, so this does not run from the management account.

ON THE EXAM

Service-managed plus automatic deployment is the combination. Without the second, every new account is a task somebody must remember.

Extending CloudFormation

- ◆ **Custom resources** call a Lambda function or SNS topic during deployment, for anything CloudFormation cannot do natively.
- ◆ They must signal success or failure, or the stack hangs until it times out. This is the classic custom resource failure.
- ◆ **Hooks** inspect a deployment before it proceeds and can block it. Control Tower's proactive controls are built on them.
- ◆ **Modules** package a reusable block of resources for use inside templates.

ON THE EXAM

A custom resource does work. A hook makes a judgement. If a scenario wants a deployment stopped, that is a hook.

Bringing existing resources under control

- ◆ **Resource import** brings an existing resource into a stack without recreating it.
- ◆ **The IaC generator** scans an account and writes a template from resources that already exist.
- ◆ **Stack refactoring** moves resources between stacks and renames logical IDs, with no replacement.
- ◆ The old retain-and-reimport dance is no longer the answer for splitting a stack. Older material still teaches it.

ON THE EXAM

Splitting a monolithic stack used to mean retain, delete, re-import. Stack refactoring does it in one operation now.

Newer safety and workflow features

- ◆ **Drift-aware change sets** show a three-way diff, so a deployment cannot silently overwrite a manual fix.
- ◆ **Git sync** deploys a stack from a repository on every commit, and posts the change set into a pull request.
- ◆ **StackSets deployment ordering** handles dependencies between accounts, such as networking before workloads.
- ◆ Sequencing and concurrency controls limit blast radius during a large rollout.

ON THE EXAM

Git sync makes the repository the source of truth. A change made in the console is drift by definition afterwards.

An honest scope note on the alternatives

- ◆ **AWS CDK** writes CloudFormation from a real programming language, and is what many teams now use.
- ◆ **AWS SAM** is a serverless-focused transform over CloudFormation. **Terraform** is the multi-cloud alternative.
- ◆ **None of the three is on the exam's in-scope service list.** CloudFormation is.
- ◆ Expect them named in scenarios as context. Expect the answer to be expressed in CloudFormation.

ON THE EXAM

This is a gap between the modern default and the examinable one. CDK compiles to CloudFormation, so the concepts transfer.

Reading the scenario

THE SCENARIO SAYS

A baseline in every account, including new ones

See what an update will change first

Someone changed a resource in the console

Split one large stack without downtime

Do something CloudFormation cannot do natively

Block a non-compliant deployment

IT IS POINTING AT

StackSets, service-managed, auto-deploy

A change set

Drift detection

Stack refactoring

A custom resource

A hook

ON THE EXAM

Separate three questions: what is being deployed, how widely, and what must not go wrong. Each has its own feature.

Meridian Freight must deploy a standard set of IAM roles, a Config recorder and a logging configuration into every account. New accounts are vended weekly into one of four OUs.

The current process is a CloudFormation stack deployed by hand after each account is created, and it has been missed twice.

What should the solutions architect recommend?

- A A StackSet with self-managed permissions, targeting the account list
- B A StackSet with service-managed permissions targeting the four OUs, with automatic deployment enabled
- C A CodePipeline that deploys the stack to each account on a schedule
- D A Config rule that detects accounts missing the resources

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

- A

 Self-managed, account list
Works, and requires roles created in every target account and the list updated for each new one. The list is the thing being forgotten.
- B

 Service-managed, OU targets, auto-deploy
Targets OUs rather than accounts, and a new account in a target OU receives the stack automatically.
- C

 Scheduled pipeline
Builds a scheduler to reproduce something StackSets does on account creation, and leaves a window between vending and baseline.
- D

 Config rule
Detective again. It reports the account that was missed, which is how they found out the last two times.

THE SIGNAL

Automatic deployment is the feature being tested. Without it, a new account is still a task somebody must remember.

A routine stack update intended to add a tag to an RDS instance instead destroyed and recreated it, losing four hours of transactions. The template change modified the DB subnet group.

The team wants to guarantee this can never happen again on any database.

What should the solutions architect implement?

- ☐ A Review change sets before every update, checking the Replacement column
- ☐ B Set UpdateReplacePolicy and DeletionPolicy to Snapshot, and add a stack policy denying updates to the database
- ☐ C Enable termination protection on the stack
- ☐ D Increase the RDS backup retention period to 35 days

◆ PAUSE AND DECIDE

What should the solutions architect implement?

A Review change sets

Good practice and essential, and it depends on a human noticing a column under time pressure. The requirement says guarantee.

C Termination protection

Protects against the stack being deleted. The stack was never deleted here; a resource inside it was replaced.

B UpdateReplacePolicy plus stack policy

The stack policy denies the update outright, and the snapshot policies protect the data if a replacement is ever deliberately allowed.

D Longer backup retention

More restore points and the same four hours lost. It shortens recovery, and does not prevent the event.

THE SIGNAL

Deletion, replacement and stack deletion are three different events. Only UpdateReplacePolicy and a stack policy address the middle one.

During an incident, an engineer widened a security group rule in the console to restore service. Two weeks later a routine stack update silently reverted it and the incident recurred. The team wants deployments to surface manual changes rather than quietly discarding them.

What should the solutions architect recommend?

- ☐ A Deny console access to security groups with an SCP
- ☐ B Run drift detection before deployments, and use drift-aware change sets to surface the difference
- ☐ C Add the security group to a stack policy denying updates
- ☐ D Disable automatic rollback so the update does not revert

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

A Deny console access

Would have prevented the emergency fix that restored service. It solves the symptom by removing the incident response.

C Stack policy on the group

Freezes the resource so CloudFormation never manages it again, which quietly makes the template wrong forever.

B Drift detection and drift-aware change sets

Exactly what was asked: surface the manual change so a human decides whether to keep it or fold it into the template.

D Disable rollback

Rollback was not involved. The update applied the template successfully; the template was simply out of date.

THE SIGNAL

Read what the team actually asked for. They wanted visibility, not prevention, and only one option provides it.

A single CloudFormation stack has grown to hold a VPC, three databases and eleven application services. Deployments now take 90 minutes and one team's change blocks everyone.

The company wants networking, data and application resources in separate stacks, with no downtime and no resource replacement.

What should the solutions architect use?

- ☐ A Set DeletionPolicy to Retain, delete the stack, then import resources into three new stacks
- ☐ B CloudFormation stack refactoring to move resources into the new stacks
- ☐ C Convert the stack into nested stacks with a parent template
- ☐ D Recreate the resources in three new stacks and migrate data across

◆ PAUSE AND DECIDE

What should the solutions architect use?

- A

Retain, delete, re-import

The pre-2025 technique, and it works. It is many manual steps with a window where nothing manages the resources.
- B

Stack refactoring

Purpose-built to move resources between stacks and rename logical IDs without replacement, in one operation.
- C

Nested stacks

Keeps one lifecycle, so one team's change still blocks everyone. That is the actual complaint.
- D

Recreate and migrate

Replaces every resource and requires data migration, contradicting both stated constraints.

THE SIGNAL

Option A was the correct answer until February 2025. Study material written before then still teaches it.

A deployment must register the new environment's hostname with a third-party monitoring vendor's API, and store the returned identifier for later stages of the same template.

There is no CloudFormation resource type for that vendor.

What should the solutions architect implement?

- ☐ A A custom resource backed by a Lambda function that calls the vendor API and returns the identifier
- ☐ B A CloudFormation hook that calls the vendor API before deployment
- ☐ C A CodePipeline stage that runs after the stack completes
- ☐ D A Systems Manager Automation runbook triggered by an EventBridge rule on stack completion

◆ PAUSE AND DECIDE

What should the solutions architect implement?

A Custom resource

Runs during the deployment and returns values other resources in the same template can reference. Both requirements met.

C Pipeline stage after

Runs after the stack finishes, so the identifier cannot be used by later resources in the same template.

B A hook

Hooks make a judgement about whether a deployment may proceed. They do not create things or return values into the template.

D Runbook on completion

Same timing problem as C, with the stack already complete before the registration happens.

THE SIGNAL

The phrase for later stages of the same template is what forces a custom resource. Anything running afterwards is too late.

A networking stack exports subnet IDs, which eleven application stacks import. The network team needs to change the subnet layout, and CloudFormation refuses to update the export. Application teams deploy on independent schedules and cannot all change at once.

What is happening, and what should the architect recommend?

- ☐ A An exported value cannot change while it is imported; publish the values to Systems Manager Parameter Store instead
- ☐ B Convert the eleven stacks into nested stacks under the networking stack
- ☐ C Delete and recreate the networking stack during a maintenance window
- ☐ D Increase the export limit with a service quota request

◆ PAUSE AND DECIDE

What is happening, and what should the architect recommend?

- A** Parameter Store instead of exports
Correct diagnosis, and it decouples the lifecycles. Consumers read the current value at deploy time with no export lock.
- B** Nested stacks
Forces eleven independent teams onto one lifecycle, which contradicts the stated constraint about independent schedules.
- C** Delete and recreate
Deleting a networking stack takes the VPC with it, along with everything the eleven applications are running in.
- D** Quota increase
There is no quota involved. The refusal is a dependency protection, not a limit.

THE SIGNAL

The export lock is a safety feature doing its job. The fix is a looser coupling, not a way around the protection.

What to carry out of this module

- ◆ **Service-managed StackSets plus automatic deployment** is the baseline pattern. OUs are the target, not account lists.
- ◆ **Deletion, replacement and stack deletion are three events.** Databases need DeletionPolicy, UpdateReplacePolicy and termination protection.
- ◆ **Stack refactoring replaced retain-and-reimport** for splitting stacks. Older material still teaches the old way.
- ◆ Custom resources do work and return values. Hooks judge whether a deployment may proceed.

ON THE EXAM

An exported value cannot change while imported. That is a protection, and the fix is looser coupling rather than a workaround.